STANISLAV KISELEVSKII

.NET SOFTWARE DEVELOPER / ENGINEER / ARCHITECT

CONTACT INFO

Email: st.kiselevskii@gmail.com	Mobile Phone: +49 176 75 494 593
Preferred communication method:	Email
Accessibility:	On-site, Remote, Hybrid.
Start New Job Time:	May 2026, after current contract completion.
Work Permit:	Germany (Blue Card)
Location:	Marienstraße 18, Ludwigshafen (Rhein), Rheinland-Pfalz, 67063

Linked In profile:	https://www.linkedin.com/in/st-kiselevskii/
GitHub profile:	https://github.com/K-S-K/CV/

SKILLS:

Platforms	Windows, Linux, MacOS, Docker, FreeRTOS, Embedded
Languages	C#, SQL, C/C++
Databases	MS SQL Server – T-SQL, DDL, DML, Performance Optimization, SP, UDF; EF Core, Dapper, MongoDB, ADO.Net, OLEDB
Development Areas	Desktop, Server-Side (Windows and Linux), Web Services, Scientific Programming, Cross-platform Development, Microservices
Technologies	Dotnet, .Net, ASP.Net, REST API, WEB Services, WinForms, WPF, Windows Services, Linux Services, Unit Testing, Dependency Injection, Razor Pages, Blazor, I2C, xUnit, XML, Json, ASN.1, Load Testing, Git
Tools	Microsoft Visual Studio, Visual Studio Code, SQL Management Studio
Approaches	Agile, Scrum, Waterflow, TDD, DDD, SOLID, OOP

INDUSTRY DOMAINS:

Aerospace	Development of the physics-based Software Digital Twin of Gaia telescope Attitude and Orbit Control System.
Electric power	Software development for billing, power network modeling.
Telecom	Software development for billing, monitoring, data format converting.
Fintech	Software development for trading automation for fiat and crypto.
Railway	Blackbox data analysis.

EDUCATION:

Ural State Technical University (Ekaterinburg, Russia),
Engineer's degree, Automated control of electric power systems, 1992-1998.

Languages: English (my current working language), Russian (native), und Ich lerne Deutsch.

JOB EXPERIENCE:

05.2024 – 04.2026 **ASTRONOMISCHES RECHEN-INSTITUT (HEIDELBERG UNIVERSITY)**

Job Purpose: Software Digital Twin development.

Responsibilities: Requirements analysis; proposing the project architecture; development of all elements of the product; bringing the ideas of scientists expressed as papers and as prototypes to the form of a tool which can be used by them for making computational experiments with modeled system.

Achievements: During working in the position, I've done next duties:

- Translating various concepts and requirements into a format appropriate for software product development.
- Organizing the development flow for bringing models and prototypes to the usable instrument.
- Development the modules from the ideas were originally prototyped in Python and Java by my colleagues and adjusted after primary implementations due to compatibility and performance requirements.
- I've designed the full architecture and implemented a physics-based software digital twin of Gaia telescope Attitude and Orbit Control System. It is a tool for testing and tuning spacecraft attitude control algorithms - the fast, configurable, production-ready framework to prove algorithms for the next-generation telescope. The parts, I've made it from are following:
  - Modeling different time systems and conversions between them.
  - Implementing the inertial rotating Dynamic/Kinematic model which can be affected by torques.
  - Modeling two nominal attitude plans, one of them is based on the inertial rotation for testing and calibration purposes, another on mission schedule for the computational experiments. Attitude contains orientation and angular velocity for every moment of mission time.
  - Modeling disturbances (micro meteoroids, clanks and solar pressure)
  - Implementing Star Catalog to handle the Sky Map. It can receive direction and window size and provide stars list in the window with those coordinates.
  - Modeling Star Tracker Module, which simulates taking a photo of stars in the specified direction, recognize constellation patterns, and returns the "measured" orientation with hardware-specific errors.
  - Modeling Focal Plane Module, which simulates star movement over the telescope CCDs, and performs angular velocity measurements with hardware-specific delays and deviations.
  - Implementing Kalman Filter, for verifying the "measured" orientation and angular velocity.
  - Modeling Controller which compares verified attitude with nominal attitude and calculates, if necessary, a torque, which is needed to correct actual attitude.
  - Modeling Micro Propulsion System, which accepts a required torque, and, knowing the geometry and power of thrusters, produces an "applied torque", which is going to the Dynamic/Kinematic model at the next iteration of the computational experiment.
  - Creating the User Interface for configuring and executing the computational experiments, observing results in graphical and tabular form, and export experimental data for the further analysis.

Technical project description: The whole project is organized in following components, which can be used separately, and can be connected as an end-user tool:

- The set of "Library" assemblies with common mathematical abstractions and infrastructure tools.
- The set of "Modules" – assemblies with implementations of different spacecraft equipment parts.
- The "Laboratory" – special module which holds all different modules together and provides communication between them.
- UI Service, which allows user to configure experiment, see the progress, browse the results and download experiment data in CSV form.
- Rest API based service which holds the Laboratory module.
- Optional separately executable service for Star Catalog which, in some experiments, consume more memory than normally can be allocated on the workstation.

Languages, tools and technologies, used in the project: C#/.Net10, Rest API, Blazor, Docker, Linux, VSCode, Microsoft Visual Studio.

The description of this project can be seen here:
https://github.com/K-S-K/CV/blob/main/Articles/36_GaiaSDT/Article.md

05.2022 – 11.2023 **EPAM SYSTEMS**

Job Purpose: Software products support for EPAM Customer.

Business project description: The software is designed for stock market assets processing.
A tremendous and well-featured desktop tool with a long history of evolvement.

Technical project description: database, server, and desktop application (Windows Forms).

Responsibilities: Support, bug fixing, and implementing new features.

Achievements:

- Fix performance issues related to computational model data preparation memory allocation. As the result, project can handle bigger time window for analysis without hanging up at memory-reallocating cycle.
- Optimized performance of several hard SQL queries. As the result, some clients became able to work without 30 second timeouts.
- Extended the exchange order completion reporting to make several-step completion be observable in reports.
- Cleaned up external references from unnecessary ones.
- Synchronized .NET framework versions of assemblies of the project
- Implemented parts of order creation parameter set extending.
- Translated some parts of the project from Visual Basic to C#.
- Fixed a lot of minor User Interface issues.
- Created tens of unit tests to increase code test coverage.

Languages, tools and technologies, used in the project: C#, T-SQL, Microsoft Visual Studio

03.2017 – 04.2022 **SOFTMEDIALAB**

Software development for different customers.
There were several projects here.

Automatic Trading System.

Project business description: Automatic trading, different trading approaches testing.

Project technical description: The project contains an exchange trading history database, exchange emulator, real exchange connector, main trading desktop program with UI, configurations manager, and several united trading algorithms implementations, which can be used with the main system.

Project Roles: Software architect, software engineer, software developer

Achievements:

- Created the database and the exchange emulator
- Created the Exchange connector, which can transparently restore context after restoring the connection after the connection was lost to external reasons
- Created the software for the trading algorithm testing on the historical trading data (RnD)
- Created the software for trading on the exchange by the implemented trading algorithm (RnD)
- Created the trading tool which can be connected to the exchange emulator as to exchange connector
- Implemented many different trading algorithms
- Developed the configurations optimizer tool
- Developed the automatic configuration adjustment subsystem based on a schedule and the market situation measurement

Responsibilities:

- Requirement collection and analysis.
- Making architectural decisions.
- Implementing all main modules.
- Delegating some research job to other developer.
- Execution the system at the tenant server
- Maintenance production, evolvement of the project

Languages, tools and technologies, used in the project: C#, T-SQL, WinForms, WPF, MS Visual Studio.

https://github.com/K-S-K/CV/blob/main/Articles/04_TDATrading/Article.md

Copy Trading.
Business project description: Providing the following trading by the follower traders after the leader trader on the Binance crypto exchange. This project automatically copies Originator trader positions for the Follower trader accounts on the Binance Cryptocurrency Exchange.

Technical project description: Technically, it is Windows Service which contains listener module and set of trader modules.

- Listener module connects to the Exchange with the Originator Trader account. It listens the signals which Originator Trader makes from their trade terminal during their normal trader's activity. Listener publishes these signals in the application's message multiplexer.
- Each of Trader modules connect to the Exchange with the particular Follower account and subscribes to Listener's signals and adds them to its own incoming signal queue. When it proceeds the signal, it makes the same trading position, as Originator trader makes, but adjust the size of the position according to the Follower's asset amount.

This is the whole idea. The service owner has some fee from the Follower traders as the percent of their effort.

Responsibilities: I was involved from the very beginning of the project. So, I've done the following:

- Collecting the primary requirements from the Customer.
- RND - Check the viability of the idea (secondary connecting to the exchange by tool with the Originator trader account, listening echo of trading orders and order update signals, creating the same orders for the Follower's account).
- Proposing the MVP architecture specification, discuss it's stages with the Customer.
- Creating the MVP, deploy to the tenant server, transfer it to the Customer.
- Customer support, collect Customer experience and wishes for the first version.
- Collecting the requirements from the Customer after theirs first experience with tool.
- Planning the sprints for the first working version development.
- Implementation.
- Technical support
- Transferring project to my colleague, code review during several following sprints.

Languages, tools and technologies, used in the project: .Net5, WPF, MS Visual Studio, Binance.Net library by JKorf, and some domain area knowledge.

The description of this project can be seen here:
https://github.com/K-S-K/CV/blob/main/Articles/27_CopyTrading/Article.md

03.2011 – 03.2017 **EASTWIND**

Software development for telecom billing systems.
It were three projects there.

EWReliability.
Project business description: Reliability analysis system for the cell operator billing system.
Practical application: To measure software's availability factor in production and the CI cycle.
Project purpose: The analysis of the program's complex reliability parameters and status.

Project technical description: Application health data collection and analysis. Availability factor measuring. Tool for investigation of the evolving of accidents. Software for load monitoring and emergencies prediction. Tool for loading testing of billing software after every sprint.

Initially, we had one point of interest: to measure the availability factor of our software (a cell operator billing system) to be sure it meets the requirements of our customer (cell operator). They asked us to provide an availability factor = 0.99995. We've researched methodology for measuring a time interval we can qualify as a "working interval" and a time interval we can call a "not working interval". The ratio of a calculated sum of the working interval to the whole observation time is the availability factor we need.

Project Roles: Software developer, Product Owner, Team leader.

Achievements:

Thanks to the project I've made, we obtain the simple interpretable metric, which show us, if the current release better or worse than previous one, and if we fit the requirements, and if our product still viable or better to not update the client.

This project helped us several times to avoid deployment of releases which where not suitable in the performance perspective and save the client's business from significant money loss.

I've made the product of the company testable, observable and measurable by performing the following steps:

- Development the technology of collecting reliability-related metrics from applications and interpretation this data according to the Reliability theory.
- Implementing the distributed tool for the collecting these metrics from the plenty of services, distributed over cluster.
- Development the database for collecting this data.
- Development the company's product deployment to the test environment.
- Development the test environment which allows deploy and configure company's product services, put configured load to the product, measure the correctness and timings of the feedback and report the results.
- Development the plenty of traffic types to load system with realistic load proportions, as it works in the production.
- Development the result representation system in tabular and graphical forms.
- Development of the production-version of the system to monitor it's working in the real time, investigate accidents, predict and alarm emergencies.

Responsibilities:

- Researched the possibilities of availability factor measuring.
- Developed a technology of software monitoring health.
- Created the DLL for other software developers to emit the health data to the custom Performance counters.
- Created the software for collecting health data in the MS SQL Database.
- Formalized the requirements for dividing operation time into the periods, classified as "working," "hanging," and "failures" (this part was implemented by another software developer whom I shared part of my job with)
- Formalized the requirements for the graphical representation of the classified periods for all observing applications on the common time scale for the developer whom I shared this part of the job with.
- Developed the software tool to view these intervals in the form of a table and the graphical form for the emergencies evolving investigation.

- Developed the software for automatic periodical reporting.
- Developed the software for generating different traffic types for the load testing of the billing software.
- Organized and performed the load testing of the billing software after every sprint (we have not approved the release if its availability factor was significantly worse than the availability factor of the previous version).
- Team management: 1 to 4 Software Developers

Evolving features of the project we implemented with wonderful people I worked with:

- Configurable deployment over the cluster subsystem.
- Many different traffic models for the load testing subsystem.
- Persistent queue of measures to survive the database server unavailability periods without keeping all measures in memory (sometimes it was extremely necessary).
- Several additional measuring subsystems like SQL Server availability observation component and MongoDB Health monitoring component, because SQL Server and MongoDB don't naturally provide data we need to observe them.
- Reliability Interval Marking on DB subsystem which is running on SQL Server by the SQL Server Agent and makes ready-to-use intervals of work for every observing software instance.
- Tree-like table interval representation for the UI and the reports.
- Reliability diagram - a scalable graphical representation of the reliability intervals on a common time scale for manual visual analysis of emergency situations evolution process.
- Dangerous situations detector on the database side - a module that finds time periods when applications are overloaded simultaneously (so-called "Pillars") to show them on the Reliability diagram and in the report tables.
- Emailing subsystem which hourly sends reports with reliability diagrams and dangerous situations tables.
- Online Diagram - the visualization of the real-time state of the observing applications.

Tools: Microsoft Visual Studio, Microsoft SQL Server Management Studio, SQL Server Profiler for bottlenecks research and for DB stuff optimizing, WinDBG for collecting the evidence of the tested software memory leaks and deadlocks.

Technologies: .Net Framework, Windows Forms, Windows Services, Application Domains, Desktop Applications, MS SQL Server (SP, UDF, Triggers, Jobs, Indexes, etc.).
Languages: C#, T-SQL (DDL, DML).

The description of the project can be seen here:
https://github.com/K-S-K/CV/blob/main/Articles/05_EWReliability/Article.md

ASN.1 Format driver generator.
Project business description: Tool for creating ASN.1 data files reading and writing drivers based on the notation.

Achievements:

- Automation of file format drivers' creation which allows to make file format driver during hour instead of several weeks.

Responsibilities: Created two assemblies of this project:

- ASNProcessor assembly (notation parsing, data model representation in the normalized form)
- ASNTools assembly (UI, displaying the data structure as a tree, generating format file driver assembly)

Project technical description:

- ASNData assembly (DLL) that provides base types and reading and writing functionality.
- ASNProcessor assembly (DLL) that provides ASN notation parsing and creating data model source code in C#.
- ASNTools assembly (desktop WinForms application), which opens ASN.1 notation file, displays its structure, creates a .Net project with ASNData assembly and source code generated by ASNProcessor assembly, and calls MSBuild to create a new DLL assembly which is a format driver for the ASN.1 notation file processed.

Languages, tools and technologies, used in the project: .Net, C#, MSBuild, MS Visual Studio, ASN.1.

EWMediation.
Project business description: Automatic data exchange between cell operators.

Project technical description: The data packet queue is processed by data converters and uploaders.

Project Role: Software developer

Responsibilities:

- Developed different format converters
- Implemented each format converter as DLL, which implements the interface required by user software
- Implemented tests for every format converter (as for mine converters as for those created before me)
- The formats were: text, XML, binary, and CSV.

Languages, tools and technologies, used in the project: .Net, C#, MS Visual Studio, Windows Services, Application domains.

10.2003 – 06.2007 **PROSOFT-SYSTEMS**

Responsibilities: Devolvement of some projects of the main product of our department

Product description: Electric power meters can provide data exchange by different hardware and software APIs. The server application collects data from the power meters and puts data into the SQL Server database. SQL Server Agent executes jobs that calculate result data from the primary data with measurement transformer factor and power grid topology history (bypass switches commutation history, measurement transformers replacement history e, etc.)

Achievements:

- Creation of the Electric Grid model, convenient for the users.
- Creation of the database which stores the model with all history of topology and parameters changing.
- Creation of the parameter recalculation system which consider the history of model changing.
- Creation of the cross-language tool which allows other developers to embed the same User Interface of the Power Grid Model within their programming language.
- Taking part in primary installation of the product at the first big client having more that 26 000 metering points.

My projects description: My application provides the possibility of proper data storing corresponding to the place on the power grid where measures were collected from. In other words, my application is the editor for creating the power network model in the database to store data in the right way to process data by jobs and retrieve data for the reporting. My application contains a T-SQL script that can actualize any version of the database to the actual version with all necessary data conversions, COM object which provides API and UI controls for the data structure editor, and for the different applications that work with data: viewers, report generators, etc., and desktop editor for the tree-shaped representation of the electric energy system.

My activities: Requirements preparation, integration with existing architecture, development of the architecture, implementing, testing, deployment to the first customer, technical support, collecting and implementing additional customer requirements after they got some user experience.

Applied technologies and knowledges: MFC SDI Application, OLE DB, T-SQL, and some subject area knowledge.
C++, MSVS, ATL COM object for data management level, MFC, MFC SDI Application, Windows API, CHM Help, SQL (MS SQL Server), and some subject area knowledge.
Developing database infrastructure and tools for editing the electric network counting scheme and solving rules calculated by MS SQL Agent. Database infrastructure to represent a hierarchical model of the part of the electric power system. User interface tools for the editing of the model (MFC). Different tools for the scheme data synchronization. The history of the scheme changes, which affects the summary power calculation (Current transformer replacement history, power meter replacement history, etc.)

https://github.com/K-S-K/CV/blob/main/Articles/03_ESphere/Article.md

04.1999 – 10.2003 **URAL STATE TECHNICAL UNIVERSITY TELEPHONE COMMUTATOR ADMINISTERING**

Achievements:

- Take part in the installation of Lucent Definity commutator.
- Handle all configuration activities with end-user and network connections.
- Development of our own call billing system based on CDR data from commutator.
- Automation of toll billing orders creation for end-users who connected to network with one external ANI.

Languages, tools and technologies, used in the project: C++, MSVS, ATL, MFC. Custom file-based data storage.

06.1998 – 04.1999 **RESEARCH INSTITUTE OF THE AUTOMATION OF RAILWAY TRANSPORT. (REMOTELY)**

Development of the railway black box analyzer.

Achievements:

- Development of project infrastructure.
- Development of project User Interface.
- Development of Graphical data representation

My part in this project was UI, graphics, and infrastructure.

Languages, tools and technologies, used in the project: C++, BC Builder, MSVS.

https://github.com/K-S-K/CV/blob/main/Articles/01_Railway_BB/Article.md